

Experiment - 1

PAGE No.	
DATE	/ /

- a) WAP to declare a class student having data members as name, roll.no; Accept and display data for one student.

```
#include <iostream>
#include <string>
using namespace std;
class student
{
    int r;
    string n;
public:
    void accept()
    {
        cout << "Enter name and roll number: ";
        cin >> n >> r;
    }
    void display()
    {
        cout << "Name = " << n << endl;
        cout << "Roll.no = " << r << endl;
    }
};

int main()
{
    student S1;
    S1.accept();
    S1.display();
    return 0;
}
```

Output -

Enter name and roll number : ABC

33

Name = ABC

Roll number = 33

b) WAP to declare a class Book having data members as id, name, price etc. Accept data for 2 books and display data of book having greater price.

```

-> #include <iostream>
#include <string>
using namespace std;
class Book;
{
    public;
    int bp, bpages;
    String n;
    void accept ()
    {
        cout << "Enter name and price! ";
        cin >> n >> bp;
        cout << "Enter pages! ";
        cin >> bpages;
    }
    void display ()
    {
        cout << "Book name = " << n << endl;
        cout << "Book price = " << bp << endl;
        cout << "Book pages = " << bpages << endl;
    }
};

int main ()
{
    Book b1, b2;
    b1.accept ();

```

```

b2.accept();
if (b1.bp > b2.bp) {
    cout << "Book with higher price:";
    b1.display();
}
else {
    cout << "Book with higher price=";
    b1.display();
}
else {
    cout << "Book with higher price=";
    b2.display();
}
return 0;
}

```

Output :

Enter name and price : ABC 50

Enter pages : 200

Enter name and price : XYZ 100

Enter pages : 300

Book with higher price :

Book name = XYZ

Book price = 100

Book pages = 300

c) WAP to declare a class time having data members as H, M & S. Accept data for one object and display total time in Sec.

```

→ #include <iostream>
#include <string>
using namespace std;
class Time
{
    int h, m, s, t;
public:
    void accept ()
    {
        cout << "Enter hours : ";
        cin >> h;
        cout << "Enter minutes : ";
        cin >> m;
        cout << "Enter seconds : ";
        cin >> s;
    }

```

```

    void display ()
    {

```

$t = (h * 3600) + (m * 60) + s;$

cout << "Total time = " << t;

```

    }
};

```

```
int main ()  
{  
    Time t1;  
    t1.accept();  
    t2.display();  
    return 0;  
}
```

Output -

Enter hours: 4
Enter minutes: 6
Enter seconds: 89
Total time: 14849

28/8

Experiment - 2

PAGE No.	
DATE	/ /

1. Write a C++ program to demonstrate the use of class & object :-

a) WAP to declare a class 'City' having data members as name and population. Accept this data for '5 cities' and display name of city having highest population

→

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

```
class City
```

```
{
```

```
public:
```

```
    std:: string name;
```

```
    int population
```

```
    void accept ()
```

```
{
```

```
    std:: cout << "City name:" << name << endl;
```

```
    cin >> name >> population
```

```
}
```

```
    void display ()
```

```
{
```

```
    std:: cout << "City name:" << name << endl;
```

```
    cout << "Population:" << population << endl;
```

```
}
```

```
};
```



```

int main ()
{
    vector<city> cities (5);
    for (int i = 0; i < 5; i++)
    {
        cout << "\n Enter data for city " << i+1 << " : " << endl;
        cities[i].accept();
    }
    city highestpopularity(city = cities[0]);
    for (int i = 1; i < 5; i++)
    {
        if (cities[i].population > Highestpopulation(city.population))
        {
            highestpopulation(city = cities[i]);
        }
    }
    cout << "\n" << "City with highest population : " << endl;
    cout << "Name : " << highestpopularity.city << endl;
    cout << "Population : " << HighestPopulation(city.population) << endl;
    return 0;
}

```

Output:-

Enter data for City 1:

Enter name and population of the city : Delhi 100

Enter data for City 2:

Enter name and population of the city : Mumbai 200

Enter data for City 3:

Enter name and population of the city : Kolkata 9000

Enter data for City 4:

Enter name and population of the city : Bangalore 1009

Enter data for City 5:

Enter name and population of the city : Chennai 10000

City with the highest population

Name : Chennai

Population : 10000

- (b) WAP to declare a class Account having data members as Account no. and balance. Accept this data for 10 accounts & give interest 10% where balance is equal or greater than 5000 & display them.



```
#include <iostream>
```

```
using namespace std;
```

```
class Account {
```

```
int acc no;
```

```
float balance;
```

```
public:
```

```
void accept()
```

```
{ cout << "Enter account number:";
```

```
cin >> acc no;
```

```
cout << "Enter balance:";
```

```
cin >> balance;
```

```
}
```

```
void addInterest()
```

```
{ if (balance >= 5000) {
```

```
balance += balance * 0.10;
```

```
}
```



```

}
void display () {
    if (balance >= 5000) {
        cout << "account no : " << accno << "balance with
            interest : " << balance << endl;
    }
}
};

int main () {
    account acc [10];
    for (int i=0; i<10; i++) {
        cout << "\n enter details for account " << i+1 << " : \n";
        acc[i].accept();
        acc[i].addinterest();
    }
    cout << "\n accounts with balance >= 5000 after adding
        interest : \n";
    for (int i=0; i<10; i++) {
        acc[i].display();
    }
    return 0;
}

```

Output :

Enter details for account 1 :
 enter account number : 12
 enter balance : 345

- enter details for account 2:

enter account number: 56

enter balance: 876

- enter details for account 3:

enter account number: 45

enter balance: 897

- enter details for account 4:

enter account number: 334

enter balance: 00897

- enter details for accounts:

enter account number: 55

enter balance: 678

- enter details for account 6:

enter account number: 666

enter balance: 7788

- enter details for account 7:

enter account number: 78

enter balance: 8888

- enter details for account 8:

enter account number: 45

enter balance: 7890

- enter details for account 9:

enter account number: 33

enter balance: 5644

DATE / /

- enter details for account 10 :
 enter account number : 33
 enter balance : 67865

- Accounts with balance ≥ 5000 after adding interest:

account no : 666 balance with interest : 8566.8
 account no : 78 balance with interest : 97776.8
 account no : 45 balance with interest : 8679
 account no : 33 balance with interest : 6208.4
 account no : 33 balance with interest : 74851.5

3) Write a program to declare a class 'staff' having data members as name & post. Accept this data for 5 staff & display names of staff who are 'HOD'.

→

```
#include <iostream>
#include <string>
using namespace std;
class staff {
    string name;
    string post;
public:
    void accept () {
        cout << "Enter name: ";
        cin >> name;
        cout << "Enter Post: ";
        cin >> post;
    }
}
```



```

void display () {
    if (post == "HOD") {
        cout << "name:" << name << endl;
    }
}

int main () {
    staff S[5];
    for (int i=0; i<5; i++) {
        cout << "\n enter details for staff" << i+1 << ": \n",
        S[i].accept ();
    }
    cout << "\n staff with post HOD: \n";
    for (int i=0; i<5; i++) {
        S[i].display ();
    }
    return 0;
}

```

Output: -

~~Enter~~ the details for staff 1
 Enter staff Name : Jiram
 Enter staff Post : HOD

Enter details for staff 2
 Enter staff name : Maria
 Enter staff ^{Post} ~~name~~ : Head

Enter the details for staff 3

Enter staff name : Jane

Enter staff Post : HOD

Enter details for staff 4

Enter staff name : Mark

Enter staff Post : HB

Enter details for staff 5

Enter staff name : jHB

Enter staff Post : Head

Staff who are HOD :

name of staff : Juan

~~name~~ of staff : Jane



28/8

Experiment-3

PAGE No.	
DATE	/ /

- 1) WAP to declare a class 'book' containing data members as book - title, author - name & price. Accept & display the info one object using a pointer to ~~start~~ that object.

→

```
#include <iostream>
```

```
using namespace std;
```

```
class book
```

```
{
```

```
public:
```

```
string book-title;
```

```
int Price;
```

```
string author-name;
```

```
void accept()
```

```
{
```

```
cout << "enter book name: ";
```

```
cin >> book-title;
```

```
cout << "enter the name of author: ";
```

```
cin >> author-name;
```

```
cout << "Price of the book: ";
```

```
cin >> Price;
```

```
}
```

```
void display()
```

```
{
```

```
cout << "book name is: " << book-title;
```

```
cout << "In author name: " << author-name;
```

```
cout << "In Price is: " << Price;
```

```
}
```

```
};
```

```

int main ()
{
    book b1;
    book * p;
    b1.accept();
    b1.display();

    return 0;
}

```

Output :-

```

Enter book name : Python
Enter name of author : Rusk
Price of the book : 999
Book name is : Python
Author name is : Rusk
Price is : 999

```

2) WAP to declare a class 'student' having data members roll-no & %. using 'this' pointer invoke member functions to accept & display this data for one object of the class.

→

```

#include <iostream>
using namespace std;
class student
{
    int roll-no;
    float percentage;
public:

```



```
void accept ()
{
```

```
    cout << "Enter student roll-no : " ;
```

```
    cin >> this->roll_no;
```

```
    cout << "Enter student percentage : " ;
```

```
    cin >> this->percentage ;
```

```
}
```

```
void display ()
```

```
{
```

```
    cout << "Roll number of student is : " << roll_no ;
```

```
    cout << " \n percentage is : " << percentage ;
```

```
}
```

```
};
```

```
int main ()
```

```
{
```

```
    Student s1;
```

```
    Student * p ;
```

```
    p = &s1;
```

```
    s1.accept();
```

```
    s1.display();
```

```
    return 0;
```

```
}
```

Output :

Enter the student roll-number : 39

Enter the student percentage : %93

Roll number of student is = 39

Percentage is = %93.

3) WAP to demonstrate use of nested class.

```
→ #include <stdio.h>  
#include <iostream>  
using namespace std;  
class student {  
    public:  
    int num;  
    class operations  
    {  
        public:  
        int square(int n)  
        {  
            return n * n;  
        }  
    }  
};
```

```
    public:  
    void accept()  
    {  
        cout << "Enter an number : ";  
        cin >> num;  
    }
```

```
    void display()  
    {  
        operations op;
```

```
        cout << "square of : " << num << " = " << op.square(num) << endl;  
        cout << "cube of : " << num << " = " << op.cube(num) << endl;
```



```
3  
3;  
int main ()  
{  
    number n1;  
    n1. accept ();  
    n1. display ();  
    return 0;  
}
```

Output :

Enter a number : 2

Square of 2 : 4

Cube of 2 : 8

Pr
28/8

Experiment - 4

PAGE No.	
DATE	/ /

Q.1 WAP to swap two numbers from same class using object as function argument. Write swap functions as member function.

→

```
#include <iostream>
using namespace std;
```

```
class number {
```

```
    int a, b;
```

```
public:
```

```
    void input () {
```

```
        cout << "Enter two numbers : ";
```

```
        cin >> a >> b;
```

```
    }
```

```
    void display () {
```

```
        cout << "a : " << a << "b : " << b << endl;
```

```
    }
```

```
    void swapnumbers (Number &obj) {
```

```
        int temp = obj.a;
```

```
        obj.a = obj.b;
```

```
        obj.b = temp;
```

```
    }
```

```
};
```

```
int main () {
```

```
    number n1;
```

```
    n1.input ();
```

```
    cout << "Before Swapping : ";
```

```
    n1.display ();
```

```
    n1.swapnumbers (n1);
```



```

    cout << "After swapping: ";
    m.display();
    return 0;
}

```

Output:

Enter two numbers: 2 4

Before swapping: a = 2, b = 4

After swapping: a = 4, b = 2

2. WAP to swap two numbers from same class using concept of friend function.

→

```

#include <iostream>

```

```

using namespace std;

```

```

class number {

```

```

    int a, b;

```

```

public:

```

```

    void input () {

```

```

        cout << "Enter two numbers: ";

```

```

        cin >> a >> b;
    }

```

```

public:

```

```

void input () {

```

```

    cout << "Enter two numbers: ";

```

```

    cin >> a >> b;

```

```

    void display () {

```

```

        cout << "a: " << a << "b: " << b << endl;

```

```

    }

```

```

friend void swapnumbers (number &obj);
}

void swapnumbers (number &obj) {
    int temp = obj.a;
    obj.a = obj.b;
    obj.b = temp;
}

int main () {
    number n1;
    n1.input();
    cout << "before swapping: ";
    n1.display();
    swapnumbers (n1);
    cout << "After swapping: ";
    n1.display();
    return 0;
}

```

Output:

Enter two numbers: 1 3

Before swapping:

a: 1, b: 3

After Swapping:

a: 3, b: 1

3. WAP to swap two numbers from different class using friend function.

→

```
#include <iostream>
```

```
using namespace std;
```

```
class class 1;
```

```
class class 2 {
```

```
int b;
```

```
public:
```

```
void input () {
```

```
cout << "Enter number for class 2:";
```

```
cin >> b;
```

```
}
```

```
void display () {
```

```
cout << "class b = " << b << endl;
```

```
}
```

```
friend void swapnumbers (class 1 & x, class 2 & y);
```

```
};
```

```
class class 1 {
```

```
int a;
```

```
public:
```

```
void input () {
```

```
cout << "Enter number for class 1:";
```

```
cin >> a;
```

```
}
```

```
void display () {
```

```
cout << "class 1 a = " << a << endl;
```

```
}
```

```
friend void swapnumbers (class 1 & x, class 2 & y);
```

```
};
```

```

wid swapnumbers ( class 1 & x , class 2 & y )
{

```

```

    int temp = x.a;

```

```

    x.a = y.b;

```

```

    y.b = temp;
}

```

```

int main () {

```

```

    class 1 obj 1;

```

```

    class 2 obj 2;

```

```

    obj 1 . input ();

```

```

    obj 2 . input ();

```

```

    cout << " Before swapping : " << endl;

```

```

    obj 1 . display ();

```

```

    obj 2 . display ();

```

```

    return 0;
}

```

Output:

Enter number for class 1: 5

Enter number for class 2: 2

Before swapping:

class 1 a = 5

class 2 b = 2

After swapping:

class 1 a = 2

class 2 b = 5

4. WAP to create two classes Result1 & Result2 which stores the marks of the student. Read the value of a marks for both class objects & compute the average of two results.

→

```
#include iostream <iostream>
```

```
using namespace std;
```

```
class Results2;
```

```
class Result1 {
```

```
    int marks;
```

```
public:
```

```
    void input() {
```

```
        cout << "Enter marks for results : ";
```

```
        cin >> marks;
```

```
    }
```

```
    friend void average (Result r1, Result r2);
```

```
};
```

```
class result 2 {
```

```
    int marks;
```

```
public:
```

```
    void input () {
```

```
        cout << "Enter marks for results 2 : ";
```

```
        cin >> marks;
```

```
    }
```

```
    friend void average (result r1, result r2);
```

```
};
```

```
void average (result r1, result r2) {
```



```
float avg = (r1.marks + r2.marks) / 2.0;  
cout << "avg marks = " << Avg << endl;
```

```
int main () {  
    result1 obj1;  
    result2 obj2;
```

```
    obj1 = input();  
    obj2 = input();
```

```
    average(obj1, obj2);  
    return 0;
```

```
}
```

Output:

Enter marks for result1: 80

Enter marks for result 2: 90

Avg marks = 85

5. WAP to find the greatest number among two numbers from two different classes using friend function.

→

```
#include <iostream>
using namespace std;
```

```
class number 1 {
    int a;
public:
    void input () {
        cout << "Enter number for classes: ";
        cin >> a;
    }
    friend void greatest (number n1, number n2);
};
```

```
class number 2 {
    int b;
public:
    void input () {
        cout << "Enter number for class 2: ";
        cin >> b;
    }
    friend void greatest (number 1 n1, number n2);
};

void greatest (number 1 n1, number 2 n2) {
    if (n1.a > n2.b)
        cout << "greatest number: " << n2.b << endl;
}
```

```
int main () {
```

```
    number 1 obj 1;
```

```
    number 2 obj 2;
```

```
    obj 1 input();
```

```
    obj 2 input();
```

```
    greatest (obj 1, obj 2);
```

```
    return 0;
```

```
}
```

Output:-

Enter number for class 1 : 45

Enter number for class 2 : 67

greatest number : 67

1. Create two classes, A and B with a private integer. Write a friend function sum() that can access private data from both the classes and return the sum.

```

→ #include <iostream.h>
using namespace std;
class class B;
class class A;
int A;
public:
void accept () {
    cout << "Enter value for A: ";
    cin >> A;
}
friend int sum (class A oA, class B oB);
} na;
class class B {
    int B;
public:
void accept () {
    cout << "Enter value for B: ";
    cin >> B;
}
friend int sum (class OA, class B OB);
} nb;
int sum (class OA, class B OB) {
    return OA.A + OB.B;
}
int main () {

```

PAGE No.	
DATE	/ /

```

na.accept ();
nb.accept ();
cout << "Sum: " << Sum (na, nb) << endl;
return 0;
}

```

Output:-

```

Enter Value for A: 3
Enter value for B: 6
Sum: 9

```

2. Write a program with a class Number that contain a private integer. Use a friend function swapnumbers (Number &, number &) to swap the private values of two number objects.

→

```
#include <stdio.h> <iostream>
```

```
using namespace std;
```

```
class Number {
```

```
int num;
```

```
public;
```

```
void accept () {
```

```
cout << "Enter value: ";
```

```
cin >> num;
```

```
}
```

```
void display () {
```

```
cout << "Value: " << num << endl;
```

```
}
```

```
friend void swapnumbers (Number & num1, Number & num2);
```



```

} n1, n2;
void swap numbers (number &, number &) {
    int temp = n1.num;
    n1.num = n2.num;
    n2.num = temp;
}

int main () {
    n1.accept ();
    n2.accept ();
    cout << "Before swap : " << endl;
    n1.display ();
    n2.display ();
    swap numbers (n1, n2);
    cout << "After swap : " << endl;
    n1.display ();
    n2.display ();
    return 0;
}

```

Output :-

Enter value : 3

Enter value : 6

Before swap

Value : 3

Value : 6

After swap :

Value : 6

Value : 3

3. Define two classes Box and Cube, each having a private volume. Write a friend function findgreater (Box, Cube) that determines which object has a larger volume.

→

```
#include <iostream>
```

```
using namespace std;
```

```
class Cube;
```

```
class Box
```

```
{
```

```
    double v;
```

```
public:
```

```
    void accept () {
```

```
        double w, l, h;
```

```
        cout << "Enter length, width, & height of the box!";
```

```
        cin >> l >> w >> h;
```

```
        v = l * w * h;
```

```
    }
```

```
    friend void findgreater (Box b, Cube c);
```

```
    b;
```

```
class cube {
```

```
    double v;
```

```
public:
```

```
    void accept () {
```

```
        double s;
```

```
        cout << "Enter side length for the cube!";
```

```
        cin >> s;
```

```
        v = s * s * s;
```

```
    }
```

```
    friend void findGreater (box B, cube C);
```

```
    C;
```



```

width findGreater (Box, Cube) {
    String res = (b.v > c.v) ? "Box" : ((c.v > b.v) ?
        "cube" : "Both equal");
    cout << "Greater volume " << res << endl;
}

int main () {
    b.accept();
    c.accept();
    findGreater (b, c);
    return 0;
}

```

Output:-

Enter l, w, h for Box : 3, 4, 3.

Enter side length for the cube : 3.

Greater volume : Box.

4. Create a class complex with real and imaginary parts as a private member use a friend function to add two complex no.'s and return the result as a new complex object.

→

```

#include <stdio.h> <iostream>

```

```

using namespace std;

```

```

class complex {

```

```

    double r, i;

```

```

public:

```

```

    void accept() {

```

```

        cout << "Enter real and imaginary parts: ";

```

```

        cin >> r >> i;

```

```

    }
}

```



```

void display () {
cout << r << " + " << i << " ; " << endl;
}

friend complex add (complex c1, complex c2);
}

complex add (complex c1, complex c2) {
    complex sum;
    sum.r = c1.r + c2.r;
    sum.i = c1.i + c2.i;
    return sum;
}

int main () {
    c1.accept ();
    c2.accept ();
    complex sum = add (c1, c2);
    cout << " The sum is : ";
    sum.display ();
    return 0;
}

```

5. Create a class student with private data members : ~~marks~~ & three subjects marks. Write a friend function calculate (Average) (student) that calculates the avg. marks -

→

```

#include <iostream>
using namespace std;
class student {
    string n;
    int m[3];
public:

```



```

void accept ()
{
    cout << "Enter student name: ";
    cin >> n;
    cout << "Enter marks from three subjects: ";
    cin >> m[0] >> m[1] >> m[2];
}

friend void calculate Avg (student s);
} s;

void calculate Avg // Average (student) {
    double avg = (s.m[0] + s.m[1] + s.m[2]) / 3;
    cout << "Student: " << s.n << endl;
    cout << "Average Marks: " << avg << endl;
}

int main () {
    s.accept ();
    calculate Average (s);
    return 0;
}

```

6. Create three classes. Alpha, beta, Gamma each with private data members. Write a single friend function. that can access all three and print their sum.

→

```

#include <iostream>
using namespace std;
class Beta;
class Gamma;
class Alpha; {

```

```

    int n; public:
        void accept ()
        {
            cout << "Enter value for Alpha : ";
            cin >> n;
        }
    friend int sum (Alpha a, Beta b, Gamma c);
} a;

class Beta {
    int n; public:
        void accept () {
            cout << "Enter value for beta : ";
            cin >> n;
        }
    friend int sum (Alpha a, Beta b, Gamma c);
} b;

int sum (Alpha a, Beta b, Gamma c) {
    return a.n + b.n + c.n;
}

int main () {
    a.accept ();
    b.accept ();
    c.accept ();

    cout << "Sum : " << sum (a, b, c) << endl;
    return 0;
}

```


7. Create a class Point with private members x and y. Write a friend function that calculates and returns the distance between two points.

```

→ #include <iostream>
using namespace std;
#include <cmath>
class Point {
double x, y;
public:
void accept () {
cout << "Enter x and y coordinates" ;
cin >> x >> y ;
}

friend double dist (Point p1, Point p2) ;
} p1, p2 ;
double dist (Point, Point) {
double dist dx = p1.x - p2.x ;
double dy = p1.y - p2.y ;
return sqrt ((dx * dx) + (dy * dy)) ;
}

int main () {
p1.accept () ;
p2.accept () ;
cout << "Distance: " << dist (p1, p2) << endl ;
return 0 ;
}

```


8. Create two classes : BankAccount and Audit BankAccount
private balance info. Write a friend function in Audit
that accesses & prints balance information for auditing.

```

#include <iostream>
using namespace std;
class BankAccount;
class Audit {
public:
    void pBal (BankAccount);
} and;
class BankAccount {
    double b; public:
    void accept ()
    {
        cout << "Enter bank acc balance: ";
        cin >> b;
    }
    friend void Audit::pBal (BankAcc);
} ba;
void Audit::pBal (BankAccount)
{
    cout << "Auditing Bank Acc balance: ₹ "
    << ba.b << endl;
}

int main () {
    ba.accept ();
    and.pbal (ba);
    return 0;
}

```

Q
1/14

Experiment - 5

- Q1) Write a program to find the sum of numbers between 1 to n using a constructor where the value of n will be passed to the constructor.

→

```
#include <iostream>
using namespace std;
```

```
class Sum
```

```
{
```

```
    int n, total;
```

```
public:
```

```
    Sum (int num) {
```

```
        n = num;
```

```
        total = (n * (n + 1)) / 2;
```

```
    }
```

```
    void display () {
```

```
        cout << "Sum of numbers from 1 to n" << n << " = " << total << endl;
```

```
    }
```

```
};
```

```
int main () {
```

```
    int n;
```

```
    cout << "Enter value of n: ";
```

```
    cin >> n;
```

```
    Sum obj (n);
```

```
    obj.display ();
```

```
    return 0;
```

```
}
```

```
Sum (int num) {
```

```
    n = num;
```

```
    for (int i = 1; i <= n; i++)
```

```
        total += i;
```

b) Write a program to declare a class "Student" having data members as name and percentage. Write a constructor to initialize these data members. Accept and display data for one student.

```

→ #include <string> <iostream>
using namespace std;
class Student
{
    string name;
    float p;
public:
    Student (string name, float p)
    {
        name = n;
        percentage = p;
    }
    void display () {
        cout << "Student name = " << name << endl;
        cout << "Percentage = " << p << endl;
    }
};

int main () {
    string name; Student s1 ("Aaryan", 90);
    float p; s1.display();

    cout << "Enter Student name : ";
    cin >> name;
    cout << "Enter percentage : ";
    cin >> p;
}

```


- WAP to find the sum of numbers from 1 to n using default constructor.

```

→ #include <iostream>
using namespace std;
class sum
{
    int num;
public:
    sum()
    {
        num = 4;
    }
    int calsum()
    {
        int i, add = 0;
        for (i = 1; i <= num; i++)
        {
            add = add + i;
        }
        cout << "In sum of number is : " << add;
    }
};

int main ()
{
    sum s1;
    s1.calsum();
    return 0;
}

```

WAP to find sum of numbers from 1 to n using copy constructor.

```
#include <iostream>
```

```
using namespace std;
```

```
class sum
```

```
{
```

```
    int num;
```

```
public:
```

```
    sum (int a)
```

```
{
```

```
        num = a;
```

```
}
```

```
    sum (int sum & s)
```

```
{
```

```
        num = s.num;
```

```
}
```

```
    int calsum ()
```

```
{
```

```
        int i, add = 0;
```

```
        for (i = 1; i <= num; i++)
```

```
{
```

```
            add = add + i;
```

```
}
```

```
        cout << "sum of numbers is " << add;
```

```
}
```

```
int main ()
```

```
{    sum s1 (5);
```

```
    s1.calsum ();
```

```
    return 0;
```

```
}
```


Q. WAP to declare class 'student' having data members name & percentage. Accept & display data for one object using default constructor.

```

→ #include <stdio.h> <iostream>
using namespace std;
class student
{
    int p;
    string name;
public:
    student()
    {
        name = "Anyan";
        p = 98;
    }
    void display()
    {
        cout << "In name of student is : " << name;
        cout << "In percentage of student is : " << p;
    }
};

int main()
{
    student s1;
    s1.display();
    return 0;
}

```

- WAP to declare a class 'student' having data members as name & percentage. Accept & display data for one object using copy constructor.

```

→ #include <iostream>
using namespace std;
class student
{
    float p;
    string n;
public:
    student (float a, string b)
    {
        p = a;
        n = b;
    }
    student (student &s)
    {
        p = s.p;
        n = s.n;
    }
    void display ()
    {
        cout << "In name of student is : " << n;
        cout << "In percentage of student is : " << p << endl;
    }
};

int main ()
{
    student s1 (98.6, "jack");
    student s2 (s1);
    s2.display ();
    return 0;
}

```


c. Define a class 'college' having data members as roll no, name, course. WAP using constructor with default value as "Computer Engineering" for course. Accept this data for two objects of class & display the data.

→ #include <iostream>

using namespace std;

class college

{

int roll_no;

string name, course;

public:

college ()

{

name = "Aryan";

roll_no = 35;

course = "Computer Engineering";

}

college (string a, int b, string c = "Computer Engineering")

{

name = a;

roll_no = b;

course = c;

}

void display ()

{

cout << "In name is : " << name;

cout << "In roll no is : " << roll_no;

cout << "In course is : " << course;

}

};


```
int main ()
```

```
{
```

```
college c1 ("Aryan", 35), c2 ("Vishal", 40);
```

```
c1.display ();
```

```
c2.display ();
```

```
return 0;
```

```
}
```

d. WAP to demonstrate constructor overloading

→ #include <iostream>

using namespace std;

class department

```
{
```

```
int nof, nos;
```

```
string h;
```

```
public:
```

```
department ()
```

```
{
```

```
h = "it";
```

```
nof = 20;
```

```
nos = 80;
```

```
}
```

```
department (string a, int b, int c)
```

```
{
```

```
h = a;
```

```
nof = b;
```

```
nos = c;
```

```
}
```

```
department (int x, int y)
```

```
{
```



```

    name = "rahul";
    nof = x;
    nos = y;
}

```

```

void display()
{

```

```

    cout << "In name is : " << n;
    cout << "In number of faculty is : " << nof;
    cout << "In number of student is : " << nos;
}

```

```

};

```

```

int main()
{

```

```

    department d1(),
    d2("william", 80, 60),
    d3(80, 75),
    d1.display(),
    d2.display(),
    d3.display(),
    return 0;
}

```

~~Q~~
u/11

Experiment - 6

PAGE NO.	
DATE	/ /

1. Single Inheritance

Create a base class called Person with attributes name and age : Derive a class Student from Person that adds an attribute rollNumber. Write function to display all details for of the student.

```
→ #include <iostream>
using namespace std;
class person
{
    String n;
    int a;
protected:
    void accept ()
    {
        cout << "Enter name : ";
        cin >> n;
        cout << "Enter age : ";
        cin >> a;
    }
    void display
    {
        cout << "Name : " << n;
        cout << "Age : " << a;
    }
};
class student : Public Person
{
    int roll;
public:
```



```

void accept () { person::accept ();
cout << "Enter roll no : ";
cin >> roll;
}

void display ()
{ person::display ();
cout << "In AgRoll no : " << roll;
}

int main ()
{
    student s;
    s.accept ();
    s.display ();
    return 0;
}

```

Output :

Enter name : Anyan
 Enter Age : 17
 Enter roll no : 35

Name : Anyan
 Age : 17
 Roll no : 35

2. Multiple Inheritance

Create two base classes Academic and Sports.

- Academic class contains marks of a student.

- Sports class contains sports score.

Create a ~~developed~~ derived class Result that inherits from both Academic and Sports. Write a function to calculate the ~~total~~ total score and display ~~details~~ details.

→

```
#include <iostream>
```

```
using namespace std;
```

```
class Academic {
```

```
{
```

```
protected:
```

```
int marks;
```

```
public:
```

```
void acceptm()
```

```
{
```

```
cout << "Enter marks : ";
```

```
cin >> marks;
```

```
}
```

```
};
```

```
class Sports
```

```
{
```

```
protected:
```

```
int score;
```

```
public:
```

```
void accepts()
```

```
{
```

```
cout << "Enter score : ";
```

```
cin >> score;
```

```
}
```

```
};
```



```
class result : public academic, public sports
{
    public:
    void display ()
    {
        int total;
        total = semMarks + score;
        cout << "In Marks : " << marks;
        cout << "In Score : " << score;
        cout << "In Total : " << total;
    }
};

int main ()
{
    result r;
    r.acceptm();
    r.accepts();
    r.display();
    return 0;
}
```

Output:

Enter marks : 30

Enter score : 50

Marks : 30

Score : 50

Total : 80

3. Multilevel Inheritance

Create a class Vehicle with attributes like brand and model. Derive a class Car from Vehicles which adds an attribute type, Further derive a class ElectricCar from Car which adds battery Capacity. Write functions to display all the details.

→

```
#include <iostream>
using std namespace std;
class Vehicle {
protected:
    string brand;
    string model;
public:
    Vehicle (string b, string m) {
        brand = b;
        model = m;
    }

    void displayVehicle () {
        cout << "Brand: " << brand << endl;
        cout << "Model: " << model << endl;
    }
};

class Car : public Vehicle {
protected:
    string type;
public:
    Car (string b, string m, string t) : Vehicle (b, m) {
        type = t;
    }
};
```



```
void displayCar () {  
    display - Vehicle ();  
    cout << "Type: " << type << endl;  
}
```

```
}
```

```
class ElectricCar : public Car {  
private:  
    int batteryCapacity;
```

```
public:
```

```
    ElectricCar (string b, string m, string t, int battery): Car(b,  
        m, t)
```

```
{
```

```
    batteryCapacity = battery;  
}
```

```
void displayElectricCar () {  
    displayCar ();
```

```
    cout << "Battery Capacity: " << batteryCapacity << "KWh" << endl;  
}
```

```
}
```

```
int main ()
```

```
{
```

```
    string brand, model, type;
```

```
    int battery;
```

```
    cout << "Enter brand: ";
```

```
    cin >> brand;
```

```
    cout << "Enter model: ";
```

```
    cin >> model;
```

```
    cout << "Enter type: ";
```

```
    cin >> type;
```

```
    cout << "Enter battery capacity: ";
```

```
    cin >> battery;
```

```

ElectricCar e1 (brand, model, type, battery);
cout << "In Electric Car Details : " << endl;
e1.displayElectricCar();
return 0;
}

```

Output:

Enter brand : BMW

Enter model : S

Enter type : SUV

Enter battery capacity : 100

Electric Car Details:

Brand: BMW

Model: S

Type: SUV

Battery Capacity : 100

4. Hierarchical Inheritance:

Create a base class Employee with attributes empID and name. Derive two classes Manager and Developer from Employee. Manager has an attribute department and Developer has an attribute programmingLanguage. Write functions to display details for both.

```

→ #include <iostream>
using namespace std;
class Employee
{
protected:
    int empID;
    string name;
public:
    Employee (int id, string n) {
        empID = id;
        name = n;
    }

    void displayEmployee() {
        cout << "Employee ID: " << empID << endl;
        cout << "Name: " << name << endl;
    }
};

class Manager : public Employee {
private:
    string department;
public:
    Manager (int id, string n, string dept) : Employee (id, n) {
        department = dept;
    }
}
    
```



```

void displayManager () {
    displayEmployee ();
    cout << "Department : " << department << endl;
}

};

class Developer : public Employee {
private:
    string programmingLanguage;
public:
    Developer (int id, string n, string lang) : Employee (id, n) {
        programmingLanguage = lang;
    }

    void displayDeveloper () {
        displayEmployee ();
        cout << "Programming Language : " << programmingLanguage << endl;
    }
};

int main ()
{
    int id;
    string name, dept, lang;
    cout << "Enter Manager ID : ";
    cin >> id;
    cout << "Enter Manager Name : ";
    cin >> name;
    cout << "Enter Department : ";
    cin >> dept;
    Manager m1 (id, name, dept);
    cout << "Manager Details : \n" << endl;
    m1.displayManager ();
    cout << "Enter Developer ID : ";
}

```


cin >> id;
 cout << "Enter Developer Name: ";
 cin >> name;
 cout << "Enter Programming Language: ";
 cin >> lang;
 Developer d1(id, name, lang);
 cout << "In Developer Details: \n";
 d1.displayDeveloper();
 return 0;

Output:

Enter Manager ID: 101
 Enter Manager name: Aryan
 Enter Department: IT

Manager Details:

Employee ID: 101

Name: Aryan

Department: IT

Enter Developer ID: 102

Enter Developer Name: Amish

Enter Programming Language: C

Developer Details:

Employee ID: 102

Name: Amish

Programming Language: C

5. Hybrid Inheritance

Combine multilevel and multiple Inheritance.

Create a base class Person with attributes name and age. Derive class Student from Person.

Create two classes Sports and Academics.

Derive class Result from Student & Sports.

Write functions to calculate and display total marks and sports score along with student details.

→ #include <iostream>

using namespace std;

class Person {

protected:

String name;

int age;

public:

Person (string n, int a) {

name = n;

age = a;

}

void displayPerson () {

cout << "Name: " << name << endl;

cout << "Age: " << age << endl;

}

}

class Student : public Person {

protected:

int rollNo;

public:

Student (string n, int a, int r) : Person (n, a) {

rollNo = r;

}


```
void displayStudent () {
    displayPerson ();
    cout << "Roll no: " << rollNo << endl;
}
```

```
}
class Academics {
protected:
    int marks;
public:
    Academics (int m) {
        marks = m;
    }
};
```

```
class Sports {
protected:
    int score;
public:
    Sports (int s) {
        score = s;
    }
};
```

```
class Result: public Student, public Academics, public Sports {
public:
```

```
    Result (string n, int a, int r, int m, int s) {
```

```
        Student (n, a, r), Academics (m), Sports (s) {}
```

```
    void displayResult ();
```

```
        displayResult ();
```

```
        int total;
```

```
        total = marks + score;
```

```
        cout << "Academic Marks: " << marks << endl;
```

```
        cout << "Sports Score: " << score << endl;
```



```

    cout << "Total: " << total << endl;
}
}
int main () {
    string name;
    int age, roll, marks, score;
    cout << "Enter name:";
    cin >> name;
    cout << "Enter Age:";
    cin >> age;
    cout << "Enter roll no :";
    cin >> roll;
    cout << "Enter Academic Marks:";
    cin >> marks;
    cout << "Enter Sports Score:";
    cin >> score;
    cout << "Student Result: \n";
    Result r1 (name, age, roll, marks, score);
    r1.displayResult();
    return 0;
}

```

Output:

Enter name : Anyan

Enter Age: 17

Enter Roll no: 35

Enter Academic Marks: 90

Enter Sports Score : 95

Student Result :

Name : Anyan

Roll no: 35

Academic Marks: 90

Sports Score : 95
Total : 185.

~~95~~
4/11

Experiment - 7

PAGE NO.	
DATE	/ /

Q WAP using function overloading to calculating area of laboratory (rectangle) & class room (square).

```
→ #include <iostream>
using namespace std;
class area {
public:
    void calc (float len, float breadth) {
        cout << "Area of laboratory : " << len * breadth << endl;
    }
    void calc (float side) {
        cout << "Area of classroom : " << side * side << endl;
    }
};

int main () {
    area a;
    cout << "Laboratory : " << endl;
    a.calc (12.5, 10.0);
    cout << "Classroom : " << endl;
    a.calc (8.0);
    return 0;
}
```


b. WAP using function overloading to calculate sum of 5 float values & sum of 10 integer values.

```

→ #include <iostream>
using namespace std;
class sum {
public:
    float sum (float a, float b, float c, float d, float e)
    { return a+b+c+d+e; }
    int sum (int arr[], int n) {
        int s=0;
        for (int i=0; i<n; i++)
            s+=arr[i];
        return s;
    }
};

int main () {
    sum s;
    cout << "Sum of 5 float no." << s.sum (1.1, 2, 3, 4.4, 5.5);
    int arr[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    cout << "Sum of 10 integers - " << s.sum (arr, 10);
    return 0;
}

```

Output:-

Sum of 5 float no.s = 16.5

Sum of 10 integers = 55

Q. WAP to implement unary operators when used with the object so that the numeric data member of class.

```
→ #include <iostream>
using namespace std;
class number {
int num;
public:
void setdata (int n) { num = n; }
void display () { cout << "Value = " << num << endl; }
void operator () { num = -num; }
};

int main () {
number n;
n.setdata (10);
cout << "Original : "; n.display ();
n;
cout << "After Negation : "; n.display ();
return 0;
}
```

Output :-

Original : Value = 10

After Negation : Value = -10

d. WAP to implement the unary ++ operator.

```

→ #include <iostream>
using namespace std;
class counter {
    int count;
public:
    void setdata (int c) { count = c; }
    void disp () { cout << "count = " << count << endl; }
    void operator ++ (int) { count++; }
};

int main () {
    counter c;
    c.setdata (5);
    cout << "After pre-increment"; c.disp();
    c++;
    cout << "Original"; c.disp();
    c++;
    cout << "After post increment!";
    return 0;
}

```

Output:

Original : count = 5

After Pre-increment : count = 6

After Post-increment : count = 7



Experiment - 8

PAGE No.	
DATE	/ /

a. Operator overloading

```
#include <iostream>
using namespace std;
class Mystring {
    string str;
public:
    void setstring () {
        cout << "Enter string: ";
        cin >> str;
    }
    void disp () {
        cout << str << endl;
    }
    Mystring operator + (Mystring s) {
        Mystring temp;
        temp.str = str + s.str;
        return temp;
    }
};

int main () {
    Mystring s1, s2, s3;
    cout << "First String ";
    s1.setstring ();
    cout << "Second String ";
    s2.setstring ();
    s3 = s1 + s2;
    cout << "Concatenated string: ";
    s3.disp ();
    return 0;
}
```

Output:-

First string:

Enter string: abc

Second string:

Enter string: xyz

concatenated string: abc xyz

b. WAP to create a base class Ilogin having data members names password. Declare accept() function virtual. Derive Email login membership login classes from Ilogin. Display login details and employee.

→

```
#include <iostream>
using namespace std;
class Ilogin {
protected:
    string name, password;
public:
    virtual void accept() {
        cout << "Enter name: ";
        cin >> name;
        cout << "Enter password: ";
    }
    virtual void display() {
        cout << "Name: " << name << "Password: " << password;
    }
};

class EmailLogin : public Ilogin {
public:
    void accept() override {
        cout << "\n Email login Details ";
        Ilogin::accept();
    }
    void display() override {
        cout << "Email login → ";
        Ilogin::display();
    }
};
```



```

class Membershiplogin : public Ilogin {
public:
    void accept() override {
        cout << "Membership login Details ";
        Ilogin::accept();
    }
    void login display() override {
        cout << "Membership login -> ";
        Ilogin::display();
    }
};

int main() {
    Emaillogin e;
    Membershiplogin m;
    e.accept();
    m.accept();
    cout << "Displaying Details : ";
    e.display();
    m.display();
    return 0;
}

```

Output: -

Email Login Details:

Enter Name: Alice

Enter password: alice123

Membership login Details:

Enter name: Bob

Enter password: bob123

Displaying login Details

Email login → Name : Alice

password : alice 123

Membership login → Name : Bob

password : bob 123

Q
4/11

Experiment - 9

PAGE NO.	
DATE	/ /

Q. WAP to copy contents of one file into another. Open "First.txt" in read (ios::in) mode & "Second.txt" in write (ios::out) mode. copy elements of first into second. Assume "First" is already created.

```
→ #include <iostream>
#include <fstream>
using namespace std;
int main() {
    ifstream fin("First.txt");
    ofstream fout("Second.txt");
    if (!fin) { cout << "First.txt not found"; return 0; }
    string line;
    while (getline(fin, line)) {
        fout << line << endl;
    }
    fin.close();
    fout.close();
    cout << "Contents copied successfully";
    return 0;
}
```

Output :-

first.txt not found

b. WAP to count digits & spaces using File handling

```
#include <iostream>
#include <fstream>
using namespace std;
int main () {
    ifstream fin ("First.txt");
    if (!fin) {cout << "File not found!\n"; return 0;}
    char ch ch;
    int digits=0, spaces=0;
    while (fin.get(ch)) {
        if (isdigit(ch)) digits++;
        if (ch==' ') spaces++;
    }
    fin.close();
    cout << "Digits : " << digits << "Spaces " << spaces;
    return 0;
}
```

Output :-

File not found!

c. WAP to count words using file handling.

```

→ #include <iostream>
#include <fstream>
#include <sstream>
using namespace std;
int main() {
    ifstream fin ("First.txt");
    if (!fin) {
        cout << "File not found!";
        return 0;
    }
    string word;
    int count = 0;
    while (fin >> word) {
        count++;
    }
    fin.close();
    cout << "Total words: " << count << endl;
    return 0;
}

```

Output:-

File not found!

d. WAP to count occurrence of a given word using file handling.

→

```
#include <iostream>
#include <fstream>
#include <sstream>
using namespace std;
int main () {
    ifstream fin ("First.txt");
    if (!fin) { cout << "File not found!"; return 0; }
    string word, target;
    int count = 0;
    cout << "Enter word to search ";
    cin >> target;
    while (fin >> word) {
        if (word == target) count++;
    }
    fin.close();
    cout << "The word " << target << " occurred " << count << " times";
    return 0;
}
```

Output:-

File not found!

Q2 no
7/11

Experiment-10

PAGE No.	
DATE	/ /

Q WAP to find sum of array elements using function template.

```
#include <iostream>
using namespace std;
template <class T>
T arrsum (T arr[], int n) {
    T sum = 0;
    for (int i = 0; i < n; i++)
        sum += arr[i];
    return sum;
}

int main () {
    int Arr1[5] = {1, 2, 3, 4, 5};
    cout << "Sum of Array : " << arrsum (Arr1, 5);
}
```

Output :

Sum of array : 15

b. WAP of square function using template specialization for square of integer no. & string. Write a specialized function string.

→

```
#include <iostream>
#include <string>
using namespace std;
template <class T>
T sq (T n) {
    return n*n;
}
template <>
string sq (string &strings) {
    return s+s;
}
int main () {
    int i = 5;
    string str = "ABC";
    cout << "Square of Integer : " << sq (i) << endl;
    cout << "Square of String : " << sq (str) << endl;
}
```

Output:-

Square of Integer : 25

Square of String : ABCABC

C. Calculator (10 operations)

→ #include <iostream>

using namespace std;

template <class T>

class calc { T num1, num2;

public:

calc (Ta, Tb) {

public:

calc (Ta, Tb) {

num1 = a;

num2 = b;

}

void disp() {

cout << "Addition : " << num1 + num2;

cout << "Subtraction : " << num1 - num2;

cout << "Multiplication : " << num1 * num2;

cout << "Division : " << num1 / num2;

cout << "Modulus : " << num1 % num2;

cout << "Square of num1 : " << num1 * num1;

cout << "Square of num2 : " << num2 * num2;

cout << "Cube of num1 : " << num1 * num1 * num1;

cout << "Cube of num2 : " << num2 * num2 * num2;

cout << "Average : " << (num1 + num2) / 2;

cout << "Maximum : " << (num1 > num2) ? num1 : num2;

cout << "Minimum : " << (num1 < num2) ? num1 : num2;

}

};

PAGE No.	
DATE	/ /

```
int main () {  
    int a, b;  
    cout << "Enter two integers: ";  
    cin >> a >> b;  
    calc {int} calc (a, b);  
    cout << "calculate: " << endl;  
    calc_disp ();  
}
```

Output:-

Enter two integers : 10 5

calculator

Addition : 15

Substraction : 5

Multiplication : 50

Division :- 2

Modulus : 0

Square of num1 : 100

Square of num2 : 25

Cube of num1 : 1000

Cube of num2 : 125

Average : 7

Maximum : 10

Minimum : 5

Q. WAP to implement push & pop methods from stack using class template.

```
→ #include <iostream>
using namespace std;
template <class T>
class stack { T arr[5]; int top;
public:
    stack() { top = -1; }
    void push(T val) { if (top == 4) cout << "Stack overflow",
    else
        arr[++top] = val;
    }
    void pop() { if (top == -1) cout << "Stack underflow";
    else
        cout << "Popped" << arr[top--] << endl; }
    void disp() { cout << "Stack elements:";
        for (int i = 0; i <= top; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main () {
    stack<int> s;
    s.push(10);
    s.push(20);
    s.push(30);
    s.display();
    s.pop();
    s.display();
}
```

Q
7/11

Experiment - 11

Q WAP to create a vector, modify the value multiply by a scalar value, display the vector.

→

```
#include <stdio.h> <iostream>
#include <vector>
using namespace std;
```

```
int main() {
    vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
    cout << "Initial vector : " << endl;
    for (int i = 0; i < 10; i++) {
        cout << v[i] << " " << endl;
    }
```

```
    cout << "Multiply by 10" << endl;
    for (int i = 0; i < 10; i++) {
        v[i] = v[i] * 10;
    }
```

```
    cout << "New vector : " << endl;
    for (int i = 0; i < 10; i++) {
        cout << v[i] << " " << endl;
    }
```

```
    return 0;
}
```



PAGE No.
 DATE / /

* Exp 11 (with iterators)

```
#include <iostream>
#include <vector>
using namespace std;
```

```
int main () {
```

```
    vector <int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
    cout << "Initial vector : " << endl;
```

```
    for (vector <int> :: iterator it = v.begin(); it != v.end();  
        ++it) {
```

```
        cout << *it << " ";
```

```
    }
```

```
    cout << endl;
```

```
    cout << "Multiply by 10" << endl;
```

```
    for (vector <int> :: iterator it = v.begin(); it !=  
        v.end(); ++it) {
```

```
        *it = (*it) * 10;
```

```
    }
```

```
    cout << "New vector : " << endl;
```

```
    for (vector <int> :: iterator it = v.begin();  
        it != v.end(); ++it) {
```

```
        cout << *it << " ";
```

```
    }
```

```
    cout << endl;
```

```
    return 0;
```

```
}
```


Experiment - 12

DATE

Q WAP to implement Stack

```
#include <iostream>
```

```
#include <stack>
```

```
using namespace std;
```

```
stack <int> stack1;
```

```
void disp() {
```

```
    stack <int> temp = stack1;
```

```
    while (!temp.empty()) {
```

```
        cout << temp.top() << " ";
```

```
        temp.pop();
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
int main() {
```

```
    int num;
```

```
    cout << "Enter a no. : " << endl;
```

```
    cin >> num;
```

```
    for (int i = 0; i < num; i++) {
```

```
        int temp;
```

```
        cout << "Enter element at position " << i+1 << endl;
```

```
        cin >> temp;
```

```
        stack1.push(temp);
```

```
}
```

```
    cout << endl;
```

```
    cout << "Top most element : " << stack1.top() << endl;
```

```
    cout << endl;
```

```
    cout << "Stack Elements (top to bottom) - - -> " << endl;
```

```
disp()  
cout<<endl;  
cout<<"Pop Function:"<<endl;  
stack1.pop();  
disp();  
stack1.pop();  
disp();  
stack1.pop();  
disp();  
}
```


Q WAP to implement STL Queue

```
#include <iostream>
```

```
#include <queue>
```

```
using namespace std;
```

```
int main () {
```

```
    queue <int> q;
```

```
    for (int i=0; i<=10; i++) {
```

```
        q.push(i*10);
```

```
    }
```

```
    cout << endl;
```

```
    cout << "front element " << q.front() << endl;
```

```
    cout << "Back element " << q.back() << endl;
```

```
    cout << endl;
```

```
    q.pop();
```

```
    cout << "After one pop, front = " << q.front() << endl;
```

```
    cout << endl;
```

```
    cout << "Queue elements (front to back :) ";
```

```
    while (!q.empty()) {
```

```
        cout << q.front() << " ";
```

```
    }
```

```
}
```

Pr
7/11